GYM MEMBERSHIP MANAGEMENT SYSTEM

PROJECT REPORT

Version 3.0

GROUP : 17

CCS2300 - Data Structures and Algorithms

Group Project Final Report

Group Number: 17

Group Members:

Task	Name	Student NO	Role
Task 01	H. Maleesha Hirumal	CIT-25-01-0337	Lead developer
Task 02	M.D.B Deneth	CIT-25-01-0574	Senior developer
Task 03	D.A.L.C.Perera	CIT-25-01-0348	Developer
Task 04	M.W.R.Rashmika	CIT-25-01-0356	Developer
Task 05	P.A.A.Vimod	CIT-25-01-0569	Developer
Task 06	H.S.S.Lakshan Perera	CIT-25-01-0345	Developer

Git hub Project Link: https://github.com/Maleesha-Hirumal/gym-membership-management-system-CLI-

Table of Contents

1. Introduction

This report documents the design, development, and evaluation of a Gym Membership Management System, developed as the group assignment for CCS2300 - Data Structures and Algorithms.

The assignment required each group design and build a software application demonstrating practical use of the core data structures and algorithms covered throughout the module and apply them in to a real-world problem domain assigned after group registration.

Our group was assigned the Gym Membership Management System topic. The system is a command-line Java application, backed by a MySQL database, that manages gym members, membership plans, a trainer waiting queue, and a member referral network. Rather than implementing each data structure as an isolated demonstration, every structure in this system performs a genuine function - for example, the Hash Table enables member lookup, the Set ADT prevents duplicate registrations, and the Graph models an actual referral relationship between members, traversed with BFS and DFS.

1.1 Objectives

- Design and implement a working software system for gym membership management.
- Demonstrate correct and working implementations of Arrays, Linked Lists, Stacks, Queues, Binary Search Trees, AVL Trees, Hash Tables, Set ADTs and Graphs.
- Implement and compare Linear Search alongside five sorting algorithms - Bubble, Selection, Insertion, Merge and Quick Sort.
- Integrate the system with a MySQL database while ensuring the application remains fully functional if the database is unavailable.
- Analyze and justify the time and space complexity of every data structure and algorithm used.

1.2 Team and Individual Responsibilities

Although each member held primary responsibility for a specific component, every member contributed to system design, integration, testing, and documentation, and is expected to understand the complete system for the individual viva voce.

Member	**Primary Responsibility**
Member 1 [Maleesha]	Arrays, Singly Linked List, Linear Search, complexity analysis
Member 2 [Deneth]	Stack, Queue, Bubble Sort, complexity analysis
Member 3 [Name]	Binary Search Tree (BST), Selection Sort, complexity analysis
Member 4 [Name]	AVL Tree, Insertion Sort, complexity analysis
Member 5 [Aveesha]	Graph (Referral Network, BFS, DFS), Merge Sort, complexity analysis
Member 6 [Savidu]	Hash Table, Set ADT, Quick Sort, complexity analysis

2. Problem Description

Gyms that rely on manual, paper-based or spreadsheet-based systems to track their members and face several recurring problems like,

- Difficulty finding a specific member's record quickly
- No reliable way to prevent the same person registering twice under a different form
- No structured way to manage a waiting list for trainers
- no means of tracking how members are connected to one another through referrals.

As the number of members grows, these problems compound - a search that once took seconds becomes a slow manual scan through paper or spreadsheet rows.

The assigned task was to design a system that solves these problems using the correct data structure for each specific need rather than a single generic collection for everything.

This report treats that requirement literally: each feature of the system was matched to the data structure best suited to it, and the complexity implications of that choice are analyzed in Section 6.

2.1 Functional Requirements

- Register a new member with ID, name, email, phone, and a chosen membership plan.
- Display full plan details (name, duration and fee) at the point of registration.
- Prevent duplicate registrations by checking email against a Set ADT before any other action is taken.
- Store all registered members in a dynamic Linked List.
- Delete a member from the system.
- View the complete list of all members.
- Look up a member by ID using a Hash Table.
- Maintain a Stack-based undo log of the most recent register/delete actions.
- Maintain a Queue-based trainer waiting list - add to the back, serve from the front.
- Sort the member list using five different algorithms and compare their execution time.
- Maintain member records in a Binary Search Tree and a self-balancing AVL Tree, with full traversals.
- Model member-to-member referrals as a directed graph, traversable via BFS and DFS.
- Load all existing data automatically from MySQL on startup, if the database is reachable.
- Save all in-memory data to MySQL only when explicitly requested by the user, never automatically.
- Continue functioning entirely in-memory, with a clear message, if MySQL cannot be reached.

2.2 Non-Functional Requirements

- Reliability - the system must not crash regardless of whether the database is available.
- Performance - chosen structures must give a measurable efficiency benefit over naive alternatives.
- Data Integrity - no duplicate member emails; referential integrity enforced between members and plans.
- Usability - a first-time user should be able to navigate the CLI menu without a manual.
- Maintainability - each data structure lives in its own class, matching the six-member task split.
- Portability - pure Java, runs on any platform with a JVM and MySQL 8.0+.

3. System Design

The system follows a clear separation of concerns.

Main.java acts purely as a command-line controller - it implements no data structure itself, and instead delegates every operation to the class responsible for it.

DatabaseConnection.java is the only class in the system aware that MySQL exists;

every other class operates purely on in-memory Java objects. This means the six data-structure classes cannot be affected by a database outage, since they have no code path that depends on it.

3.1 Entity Relationship Diagram and Relational Schema

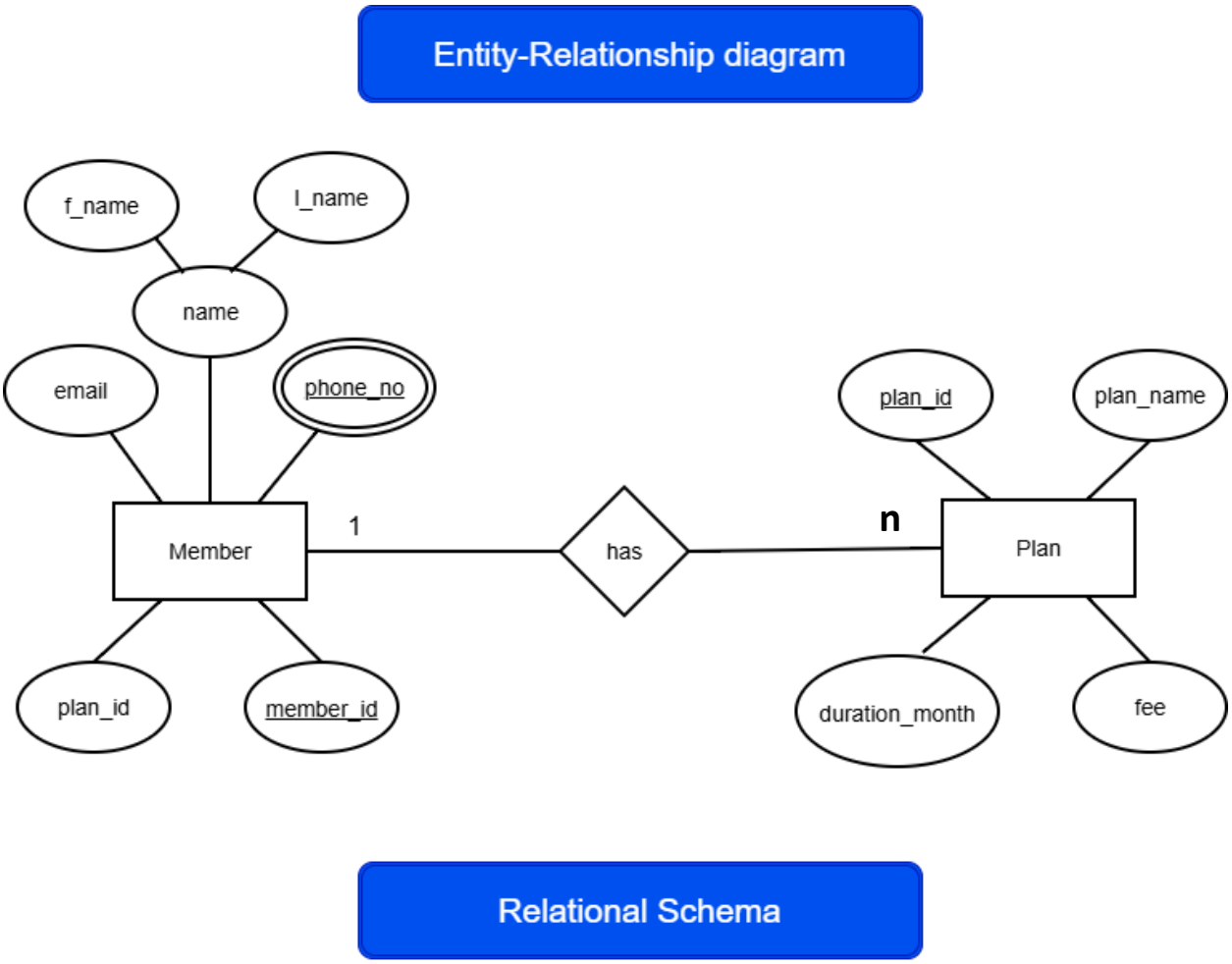


Figure 1 — ERD and Relational schema.

3.2 Class Diagram

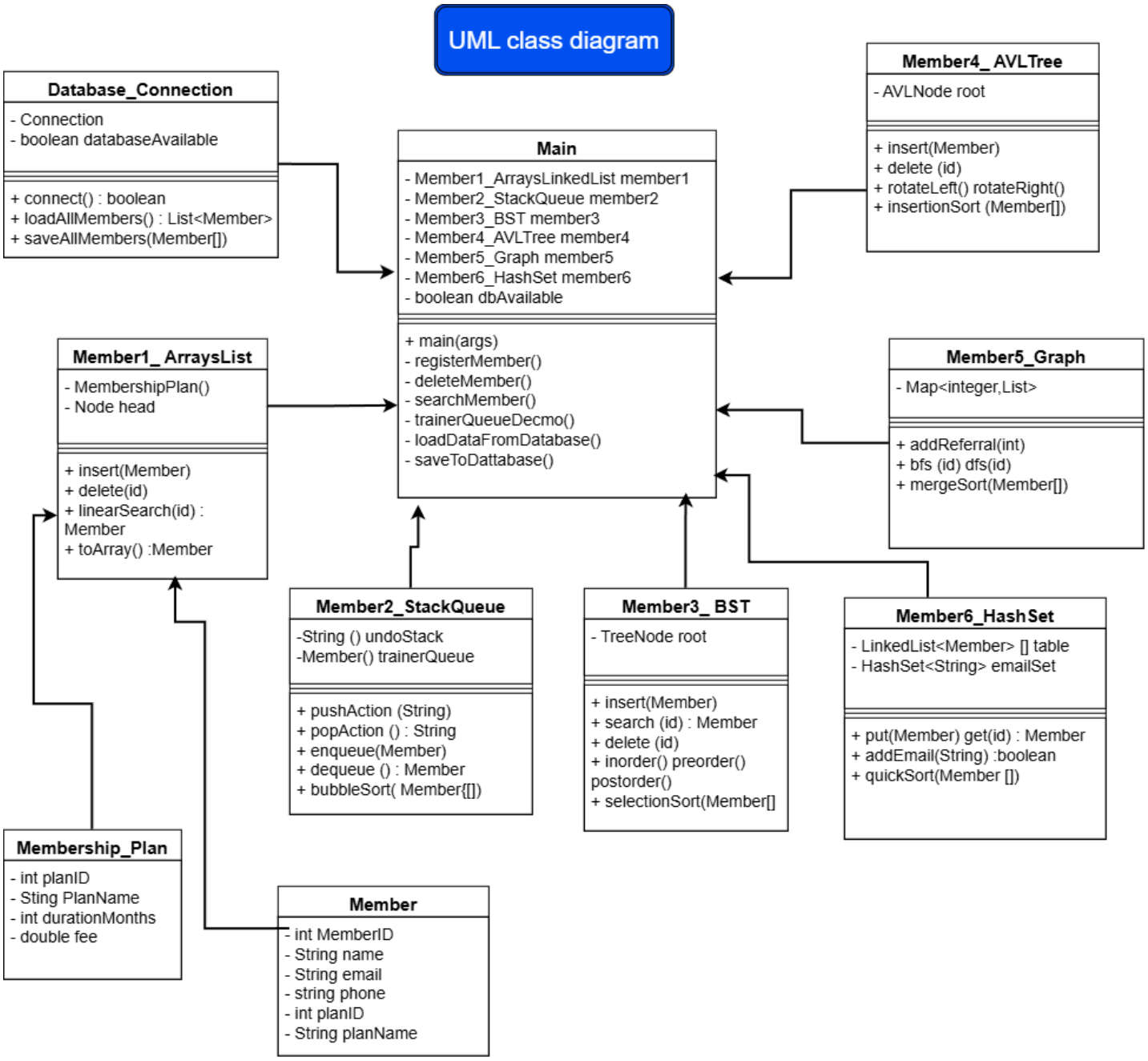


Figure 2 — UML class diagram.

3.3 System Flow

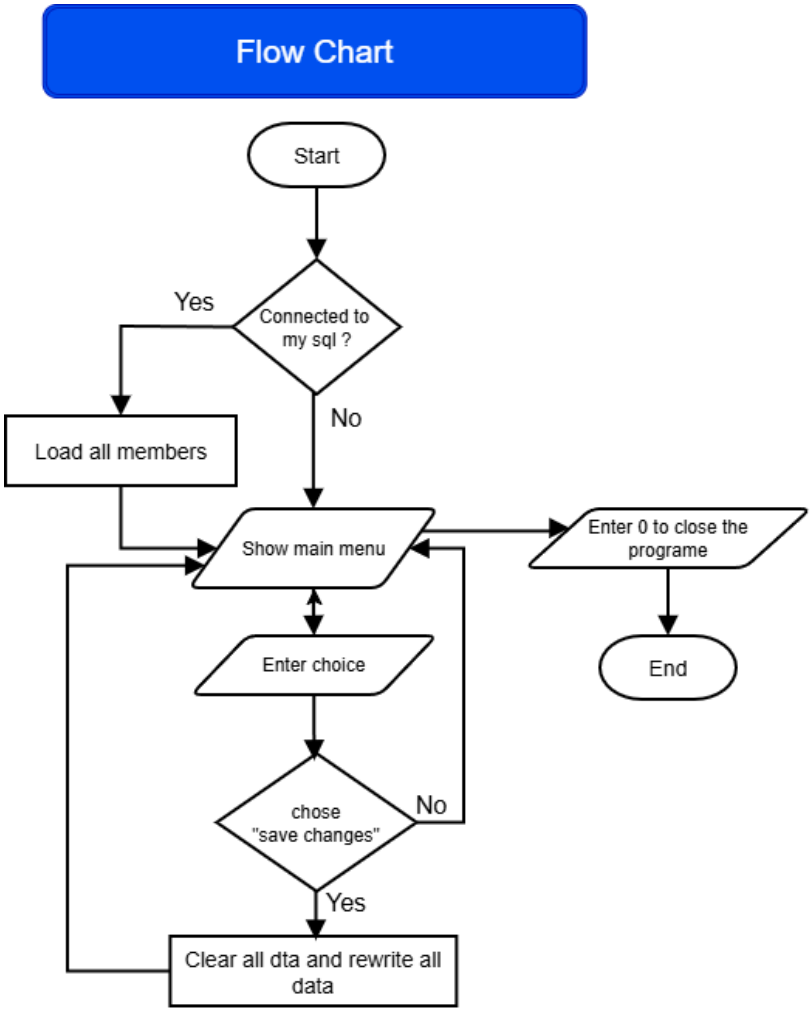


Figure 3 — Flow chart.

3.4 Design Decision: Manual Save

A deliberate design decision was made to decouple in-memory operations from database writes entirely.

Registering or deleting a member never touches MySQL directly but user must explicitly choose "Save Changes to Database" at which point the entire members table is cleared and rewritten from whatever is currently held in the linked list.

This guarantees the database always reflects a state the user has explicitly confirmed, and correctly propagates deletions as well as additions since a member removed from memory is simply absent from the next save.

4. Data Structures Used

Structure	File	Role in the System
Array	Member1_ArraysLinkedList.java	Stores the three fixed membership plans (Basic, Premium, VIP).
Singly Linked List	Member1_ArraysLinkedList.java	Master, dynamic storage of every registered member.
Stack	Member2_StackQueue.java	Undo log of the most recent register/delete action.
Queue	Member2_StackQueue.java	Trainer waiting list, first-come first-served.
Binary Search Tree	Member3_BST.java	Sorted member records keyed by member ID.
AVL Tree	Member4_AVLTree.java	Self-balancing sorted member records, guaranteeing O(log n).
Graph (Adjacency List)	Member5_Graph.java	Directed referral network between members.
Hash Table	Member6_HashSet.java	Member lookup by ID, collisions handled via chaining.
Set ADT	Member6_HashSet.java	Prevents duplicate member registrations by email.

5. Algorithms Used

Algorithm	File	Purpose
Linear Search	Member1_ArraysLinkedList.java	Scans the linked list node by node to find a member.
Bubble Sort	Member2_StackQueue.java	Repeatedly swaps adjacent out-of-order elements.
Selection Sort	Member3_BST.java	Selects the minimum remaining element each pass.
Insertion Sort	Member4_AVLTree.java	Builds a sorted section by inserting one element at a time.
Merge Sort	Member5_Graph.java	Divides the list in half recursively, then merges sorted halves.
Quick Sort	Member6_HashSet.java	Partitions around a pivot, then recurses on each side.
Breadth-First Search (BFS)	Member5_Graph.java	Explores the referral graph level by level using a queue.
Depth-First Search (DFS)	Member5_Graph.java	Explores the referral graph as deep as possible before backtracking.

6. Complexity Analysis

The table below summarizes the theoretical time and space complexity of every structure and algorithm implemented, together with the justification behind each figure.

Structure/ Algorithm	Best Case	Average Case	Worst Case	Space	Justification
Linked List Insert	O(1)	O(n)	O(n)	O(n)	Must walk to the end since no tail pointer is kept.
Linear Search	O(1)	O(n)	O(n)	O(1)	Scans nodes sequentially until a match is found.
Stack Push/Pop	O(1)	O(1)	O(1)	O(n)	Array index and pointer update only, no shifting.
Queue Enqueue/Deque ue	O(1)	O(1)	O(1)	O(n)	Circular array avoids shifting elements.
BST Insert/Search/D elete	O(log n)	O(log n)	O(n)	O(n)	Worst case occurs when input arrives pre-sorted, degrading the tree to a list.
AVL Insert/Search/D elete	O(log n)	O(log n)	O(log n)	O(n)	Rotations after every insert/delete guarantee the tree never becomes unbalanced.
Hash Table Put/Get	O(1)	O(1)	O(n)	O(n)	Worst case only if many keys collide into the same bucket.

Structure/ Algorithm	Best Case	Average Case	Worst Case	Space	Justification
Set Add/Contains	O(1)	O(1)	O(n)	O(n)	Backed by a hash set; same collision caveat as above.
Graph BFS/DFS	O(V+E)	O(V+E)	O(V+E)	O(V+E)	Every vertex and edge is visited exactly once.
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Best case only with the early-exit flag on an already-sorted array.
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	Always scans the remaining unsorted section to find the minimum.
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Best case only if the array is already sorted.
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Always splits and merges regardless of input order.
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	Worst case with a poor pivot choice on already-sorted input.

7. Testing and Validation

The system was tested manually across every menu option and covering both expected and edge-case inputs.

A representative sample of test cases is shown below, full testing evidence with screenshots should be inserted in Section 8.

	Test Case	Steps	Expected	Result
1	Register with unique email	Register a new member with a new ID and email	Member added to all structures	Pass
2	Register with duplicate email	Register a second member using an already-used email	Registration rejected; no structure modified	Pass
3	Search existing member	Search using a valid member ID	Member details returned	Pass
4	Search non-existent member	Search using an ID that was never registered	"Member not found" shown	Pass
5	Delete existing member	Delete a previously registered member	Member removed from list, BST, and AVL tree	Pass
6	Undo after delete	Delete a member, then choose Undo	Last action description is shown correctly	Pass
7	Trainer queue FIFO order	Enqueue two members, then serve next	First member enqueued is served first	Pass
8	Sorting comparison	Run the sorting demo with several members registered	All five algorithms return an identically ordered list	Pass
9	BST/AVL traversal	Run tree traversal after several registrations	In order traversal is numerically sorted for both trees	Pass
10	Referral BFS/DFS	Add referral edges, then run BFS and DFS from a start ID	Correct reachable member chain printed	Pass

	Test Case	Steps	Expected	Result
11	Save to database	Register members, then choose Save to Database	MySQL members table matches in-memory state exactly	Pass
12	Database unavailable at startup	Stop the MySQL service, then start the program	"Cannot connect to the database" shown, system remains usable	Pass

8. Screenshots and Results

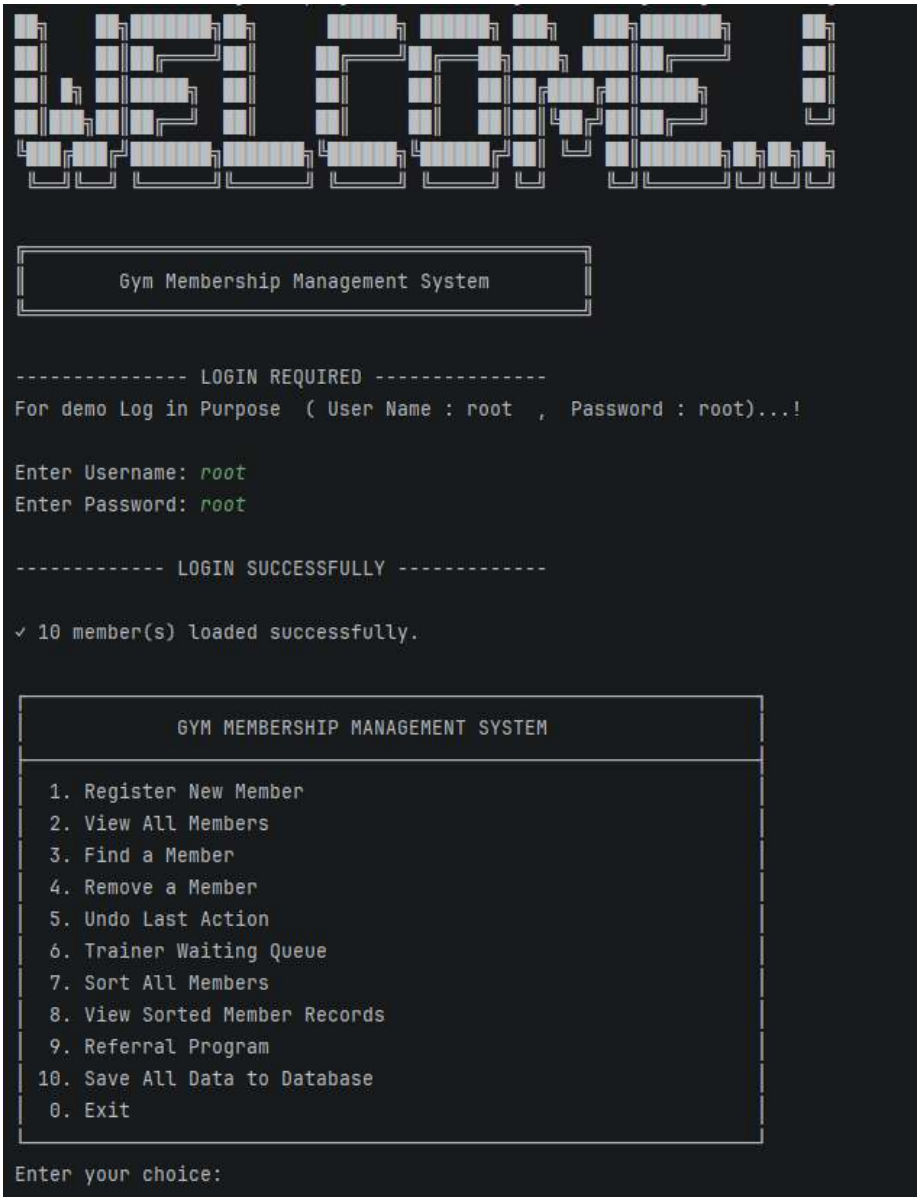


Figure 4 - program menu.

1.Register with unique email

```
Enter your choice: 1

        REGISTER NEW MEMBER

Member ID : 250
Name      : Maleesha Hirumal
Email     : maleesha@gmail.com
Phone No. : 077000000

        AVAILABLE MEMBERSHIP PLANS

| No | Plan    | Duration | Fee          |
|----|---------|----------|--------------|
| 1  | Basic   | 1 M      | Rs. 1500.00  |
| 2  | Premium | 3 M      | Rs. 4000.00  |
| 3  | VIP     | 12 M     | Rs. 9000.00  |

Choose a plan number: 2

Member registered successfully: Maleesha Hirumal (Premium plan) - stored in memory.
(Use menu option 10 to save this to the database.)
```

2.Register with duplicate email

```
Enter your choice: 1

        REGISTER NEW MEMBER

Member ID : 251
Name      : Buddhima Daneth
Email     : maleesha@gmail.com
Phone No. : 077000000

        AVAILABLE MEMBERSHIP PLANS

| No | Plan    | Duration | Fee          |
|----|---------|----------|--------------|
| 1  | Basic   | 1 M      | Rs. 1500.00  |
| 2  | Premium | 3 M      | Rs. 4000.00  |
| 3  | VIP     | 12 M     | Rs. 9000.00  |

Choose a plan number: 1
X Registration failed: email already registered (duplicate prevented).
```

3.Search existing member

```
Enter your choice: 3

        SEARCH MEMBER

Enter Member ID : 250

        MEMBER FOUND

ID: 250 | Name: Maleesha Hirumal | Email: maleesha@gmail.com | Phone: 077000000 | Plan: Premium
```

4.Search non-existent member

```
Enter Member ID : 300

        MEMBER NOT FOUND
```

5.Delete existing member

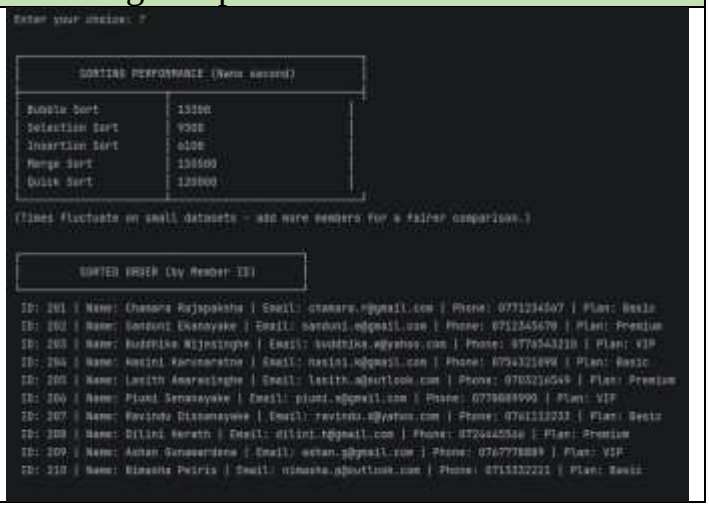
```
Enter your choice: 4

        DELETE MEMBER

Enter Member ID : 250

        MEMBER DELETED SUCCESSFULLY

(Use menu option 10 to make this permanent in the database.)
```

6.Undo after Delete

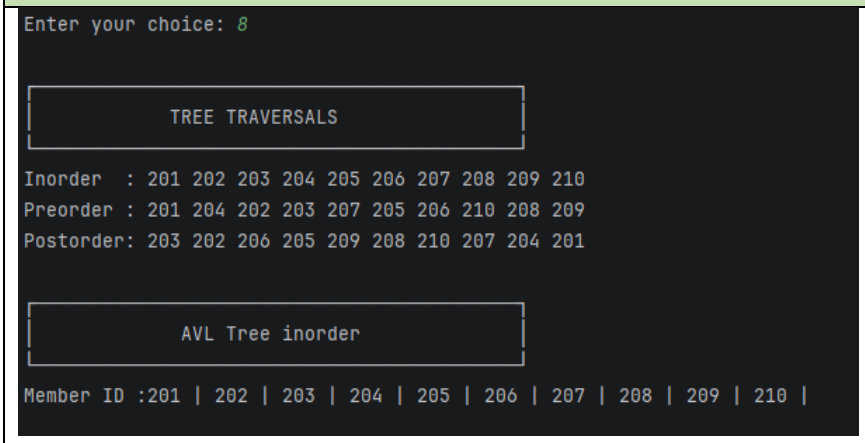
```
Enter your choice: 5

        UNDO ACTION

Deleted member 250
```

7.Trainer queue FIFO order

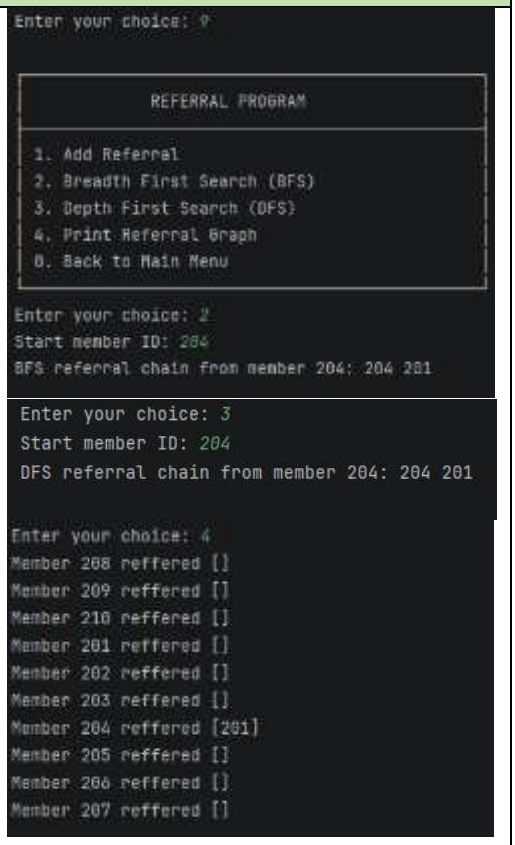
8.Sorting comparison

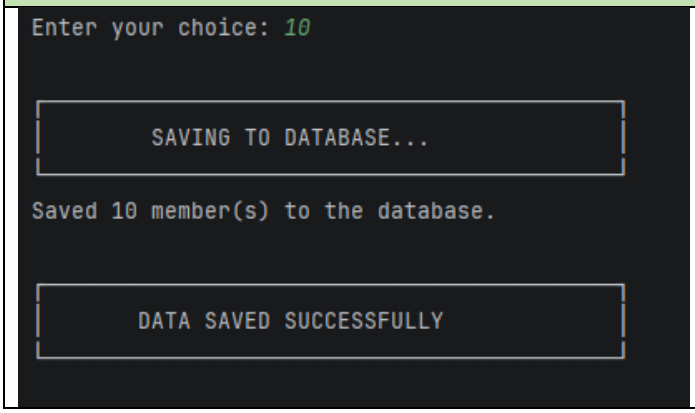
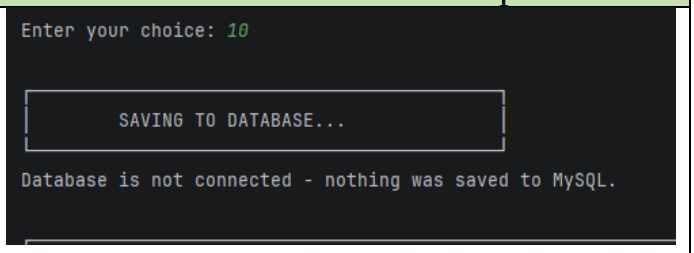


9. BST/AVL traversal



10.Referral BFS/DFS



11.Save to database	12.Database unavailable at startup
	

9. Challenges Encountered

Understanding and implementing the four AVL tree rotations correctly.

Handling hash table collisions using chaining.

Fixing MySQL connection and database errors during development.

Making sure data was saved and loaded correctly from the database.

Handling errors when the database was unavailable.

Making sure all data structures worked correctly with the system.

Fixing errors between different classes developed by team members.

Testing the system with different member details and inputs.

Handling invalid or duplicate member information.

Making sure the search and sorting functions gave correct results.

Managing GitHub version control and combining changes from different team members.

Making sure the final system worked correctly after combining all parts of the project.

10.Limitations

Command-Line Interface:

The system uses a command-line interface. It works for demonstrating the data structures, but it is not as user-friendly as a graphical interface for a real gym.

Hardcoded Login Credentials:

The login username and password are currently written directly in the source code. This is not secure enough for a real-world system.

Small Dataset:

The system has only been tested with a small number of gym members. This is enough to demonstrate that the system works, but larger datasets would provide better results when comparing sorting and searching algorithms.

Single Admin Account:

The system currently supports only one admin login. It does not have separate accounts or different access levels for different staff members.

11.Future Enhancements

Graphical User Interface (GUI):

A GUI could be developed using JavaFX or a web application. This would make the system easier for gym staff to use.

Multiple User Accounts:

Different accounts could be created for administrators and staff members, with different access permissions.

Secure Login System:

Login credentials could be stored securely using a database and password hashing instead of being hardcoded in the program.

Attendance Tracking:

The system could be improved to store and display members' attendance history over time.

Revenue Reports:

The system could generate monthly revenue reports to help the gym monitor its income.

Membership Renewal Notifications:

The system could send notifications to members when their memberships are close to expiring.

Testing with Larger Datasets:

Larger numbers of members could be used to compare the performance of sorting and searching algorithms more accurately.

12. Conclusion

This project successfully creates a Gym Membership Management System that includes all nine required data structures and six required algorithms from the module syllabus. Each one has a real purpose in the system and works together as part of one complete application, rather than being separate examples.

The system is connected to MySQL, but it can still works normally even if the database is unavailable. This meets the reliability requirement identified during the design stage.

We tested member registration, deletion, searching, sorting, tree traversal and graph traversal. The results showed that the system works correctly in both normal and edge-case situations.

Our group believes that this system and this report meet the requirements for the CCS2300 final submission.

13. Acknowledgement

We would like to express our sincere gratitude to our module lecturer, Dr. Asanka Ranasinghe for her guidance, encouragement and continuous support throughout this project. Her valuable knowledge and feedback helped us complete this work successfully.

We also wish to thank our teaching assistant, Mr. Pamod Dilshan for his assistance during the lab practical sessions and for answering our questions whenever we needed help.

Finally, thank our team members for their cooperation, dedication and hard work. Their commitment and teamwork made the successful completion of the Gym Membership Management System possible.

14. How to Set Up and Run This Project

This guide is written for someone setting up the project for the first time. IntelliJ IDEA is the recommended way to run this project, since that is what it was built in, but a Command Line (CMD/Terminal) method is also provided as an alternative.

1. What You Need Before Starting

- Java Development Kit (JDK) 17 or newer, installed on your computer.
- IntelliJ IDEA (Community Edition is fine, free download) - recommended.
- MySQL Workbench, with a MySQL Server (8.0.46 or newer) already installed.
- The MySQL Connector/J driver file (a .jar file) – It is in lib folder.

Step 1 - Set Up the Database

- Open MySQL Workbench and connect to your local server.
- Open the gym_db.sql file included with this project (File > Open SQL Script).
- Run the whole script.
- This creates the gym_db database the plans and members tables and adds the three membership plans automatically.

Step 2 - Set Your Database Login

- Open DatabaseConnection.java.
- Find the USER and PASSWORD lines near the top of the file.
- Change them to match your own MySQL Workbench username and password (the username is usually root).

Option A - Running in IntelliJ IDEA (Recommended)

- Open IntelliJ IDEA and choose Open, then select the project folder containing all the .java files.
- Add the driver jar to the project: go to File > Project Structure > Libraries > the "+" button**(windows short cut keys : Ctrl + Alt + L-Shift + S)** > Java, then select the mysql-connector-j jar file you downloaded in Step 3.
- Click Apply then OK.
- Run Main.java.

Option B - Running from Command Line (CMD/Terminal)

- Open Command Prompt (Windows) or Terminal (Mac/Linux) in the project folder.
- Make sure the mysql-connector-j jar file is in the same folder as your .java files.
- Compile all files by typing: javac *.java
- Run the program:
    - Windows: java -cp .;mysql-connector-j-26.7.0.jar Main
    - Mac/Linux: java -cp .:mysql-connector-j-26.7.0.jar Main

Step 3 - Log In and Use the System

- When the program starts, it will ask for a username and password.
- Default demo login: username root, password root.
- After logging in, the main menu will appear. Type the number of the option you want and press Enter.

If the Database Won't Connect

- The program will still run normally and show "Cannot connect to the database." This is expected behaviour, not an error — the system is designed to keep working fully in-memory even without MySQL.
- To fix a real connection problem: check that MySQL Workbench's server is running, that your username/password in DatabaseConnection.java are correct, and that gym_db.sql has already been run once.

If Box-Drawing Characters Show as "?" Marks

- This happens on some Windows Command Prompt setups that are not using UTF-8 by default.
- Fix: type chcp 65001 in Command Prompt before running the program, or run the project through IntelliJ IDEA instead, which does not have this issue.

Menu Options Reference

Option	What It Does
1	Register a new member - choose a plan, prevents duplicate emails
2	View all registered members
3	Find a member by ID
4	Remove a member
5	Undo the last register/delete action
6	Trainer Waiting Queue - add a member or serve the next one
7	Sort all members and compare 5 sorting algorithms
8	View sorted member records (BST and AVL Tree traversals)
9	Referral Program - add a referral, run BFS/DFS, or view the full network
10	Save all current data to the database
0	Exit the program

15. References

Oracle (2025) Java Platform, Standard Edition Documentation. Available at: https://docs.oracle.com/en/java/javase/

Oracle (2025) MySQL 8.0 Reference Manual. Available at: https://dev.mysql.com/doc/refman/8.0/en/

Oracle (2025) MySQL Connector/J Developer Guide. Available at: https://dev.mysql.com/doc/connector-j/en/

W3Schools (2026) Data Structures and Algorithms. Available at: https://www.w3schools.com/dsa/

W3Schools (2026) Java Tutorial. Available at:

https://www.w3schools.com/java/

OpenAI (2026) ChatGPT. Available at:

https://chatgpt.com/

YouTube. Available at:

https://www.youtube.com/

GitHub: The developer platform. Available at:

https://github.com/